Asignatura: Estructuras de Datos
Duración estimada: 2 horas (120 min)
Modalidad: Individual
Nombre del estudiante: Noemi Mata Ruiz **Fecha:** Viernes 30 Octubre 2026

1. Objetivos de aprendizaje

Al finalizar esta práctica, el estudiante será capaz de:

1. Explicar el principio FIFO y las operaciones básicas de una cola.
2. Implementar una cola con **arreglo circular**.
3. Implementar una cola con **nodos enlazados** usando genéricos.
4. Utilizar la interfaz `Queue` de la librería estándar de Java.
5. Aplicar colas para resolver un problema de simulación.

2. Requisitos previos

• Conocimientos básicos de Java: clases, objetos, arreglos, ciclos y excepciones. • Haber visto el concepto de nodo y referencias (para la Parte 3).
• JDK 11 o superior instalado.
• VS Code con extensión de Java).

3. Resumen teórico

Una **cola** es una estructura lineal que sigue el principio **FIFO** (*First In, First Out*): el primer elemento en entrar es el primero en salir.

Operación	Descripción	Complejidad
`encolar`	Agrega un elemento al final	O(1)
`desencolar`	Elimina y devuelve el elemento del frente	O(1)
`frente`	Consulta el frente sin eliminarlo	O(1)
`estaVacia`	Indica si no hay elementos	O(1)
`tamanio`	Devuelve la cantidad de elementos	O(1)

```
Frente Final
 ↓ ↓
[ 10 | 20 | 30 | 40 ] ← encolar(50)
 ↑
desencolar() → sale el 10
```

4. Preparación del entorno

Paso 1. Crea un nuevo proyecto Java llamado `LabColas_[nombre]`.

Paso 2. Dentro de `src`, crea el paquete `colas`.

Paso 3. Verifica que todo funciona creando la clase `Main` con este código y

ejecutándola: `package colas;`

```
public class Main {
 public static void main(String[] args) {
 System.out.println("Entorno listo");
 }
}
```

Punto de control: la consola muestra `Entorno listo`.

```
C:\Estructura de Datos>git clone git@github.com:noemi-mata/LabColas_NoemiMataRuiz.git
Cloning into 'LabColas_NoemiMataRuiz'...
Enter passphrase for key '/c/Users/Noemi/.ssh/id_ed25519':
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (3/3), done.

C:\Estructura de Datos>cd LabColas_NoemiMataRuiz

C:\Estructura de Datos\LabColas_NoemiMataRuiz>mkdir src/colas
La sintaxis del comando no es correcta.

C:\Estructura de Datos\LabColas_NoemiMataRuiz>mkdir src\colas

C:\Estructura de Datos\LabColas_NoemiMataRuiz>cd src\colas

C:\Estructura de Datos\LabColas_NoemiMataRuiz\src\colas>notepad Main.java

C:\Estructura de Datos\LabColas_NoemiMataRuiz\src\colas>cd ..

C:\Estructura de Datos\LabColas_NoemiMataRuiz\src>javac colas\Main.java

C:\Estructura de Datos\LabColas_NoemiMataRuiz\src>java colas.Main
Entorno listo

C:\Estructura de Datos\LabColas_NoemiMataRuiz\src>
```

Parte 1. Cola con arreglo circular

En un arreglo normal, al desencolar quedan espacios vacíos al inicio que no se reutilizan. En un arreglo **circular**, los índices "dan la vuelta" usando el operador módulo (`%`).

Paso 1.1. Crear la clase y sus atributos

Crea la clase `ColaArreglo` en el paquete `colas`, documenta el Código descripción de la

clase `package colas;`

```
public class ColaArreglo {
```

```
 private int[] datos;
 private int frente;
 private int fin;
 private int tamanio;
 private int capacidad;

 public ColaArreglo(int capacidad) {
 this.capacidad = capacidad;
 this.datos = new int[capacidad];
 this.frente = 0;
 this.fin = -1;
 this.tamanio = 0;
 }
}
```

Paso 1.2. Métodos de consulta

Agrega dentro de la clase:

```
public boolean estaVacia() {
 return tamanio == 0;
}

public boolean estaLlena() {
 return tamanio == capacidad;
}

public int tamanio() {
 return tamanio;
}
```

Paso 1.3. Método `encolar`

```
public void encolar(int valor) {
 if (estaLlena()) {
 throw new IllegalStateException("La cola está llena");  }
 fin = (fin + 1) % capacidad; // avance circular
 datos[fin] = valor;
 tamanio++;
}
```

Paso 1.4. Métodos `desencolar` y `frente`

```
public int desencolar() {
 if (estaVacia()) {
 throw new IllegalStateException("La cola está vacía");  }
 int valor = datos[frente];
 frente = (frente + 1) % capacidad;
 tamanio--;
 return valor;
}

public int frente() {
 if (estaVacia()) {
 throw new IllegalStateException("La cola está vacía");  }
```

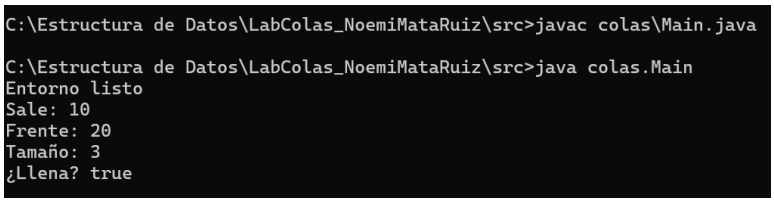
```
 return datos[frente];
}
```

Paso 1.5. Probar la cola

En `Main`, escribe esta prueba que fuerza el "giro" del arreglo:

```
ColaArreglo cola = new ColaArreglo(3);
cola.encolar(10);
cola.encolar(20);
cola.encolar(30);
System.out.println("Sale: " + cola.desencolar()); // 10
cola.encolar(40); // reutiliza la posición 0
System.out.println("Frente: " + cola.frente()); // 20
System.out.println("Tamaño: " + cola.tamanio()); // 3
System.out.println("¿Llena? " + cola.estaLlena()); // true
```

Punto de control: la salida debe ser `Sale: 10`, `Frente: 20`, `Tamaño: 3`, `¿Llena? true`.



Preguntas de análisis (Parte 1)

1. ¿Qué valor tiene `fin` después de encolar 40? ¿Por qué?

   Vale 0. Después de encolar 10, 20 y 30, fin quedó en 2. Al encolar 40 se calcula (2 + 1) % 3 = 0, así que el índice da la vuelta y el 40 se guarda en la posición 0, que había quedado libre cuando salió el 10.

2. ¿Qué pasaría si usáramos `fin + 1` sin el módulo?

   fin valdría 3, que es una posición fuera del arreglo (solo existen 0, 1 y 2), y el programa fallaría con ArrayIndexOutOfBoundsException. Además, los espacios que se liberan al inicio al desencolar nunca se volverían a usar.

3. ¿Qué ocurre al llamar `encolar` con la cola llena? ¿Cómo lo manejaría una versión que crece dinámicamente?

   Se lanza una IllegalStateException con el mensaje “La cola está llena”. Una versión dinámica crearía un arreglo nuevo del doble de tamaño. Copiaría los elementos en orden empezando desde frente, con el índice (frente + i) % capacidad, y después pondría frente = 0 y fin = tamanio - 1.

Parte 2. Cola con nodos enlazados

Idea clave: esta cola no tiene límite de capacidad. Se mantienen dos referencias: `frente` (donde

se elimina) y `fin` (donde se inserta).

Paso 2.1. Crear la clase con el nodo interno, es necesario documentar cada clase, propiedades y método

```
package colas;
public class Nodo {
 T dato;
 Nodo siguiente;

 Nodo(T dato) {
 this.dato = dato;
 }
}
```

```
public class ColaEnlazada<T> {

 private Nodo frente;
 private Nodo fin;
 private int tamanio;
}
```

Paso 2.2. Métodos `estaVacia` y `tamanio`

```
public boolean estaVacia() {
 return frente == null;
}

public int tamanio() {
 return tamanio;
}
```

Paso 2.3. Método `encolar`

Piensa en los dos casos: cola vacía y cola con elementos.

```
public void encolar(T valor) {
 Nodo nuevo = new Nodo(valor);
 if (estaVacia()) {
 frente = nuevo;
 } else {
 fin.siguiente = nuevo;
 }
 fin = nuevo;
 tamanio++;
}
```

Paso 2.4. Método `desencolar`

```
public T desencolar() {
 if (estaVacia()) {
```

```
 throw new IllegalStateException("La cola está vacía");  }
 T valor = frente.dato;
 frente = frente.siguiente;
 if (frente == null) {
 fin = null; // la cola quedó vacía
 }
 tamanio--;
 return valor;
}
```

Paso 2.5. Método `frente`

```
public T frente() {
 if (estaVacia()) {
 throw new IllegalStateException("La cola está vacía");  }
 return frente.dato;
}
```

Paso 2.6. Probar con genéricos

```
ColaEnlazada<String> nombres = new ColaEnlazada<>();
nombres.encolar("Ana");
nombres.encolar("Luis");
nombres.encolar("María");
System.out.println(nombres.desencolar()); // Ana
System.out.println(nombres.frente()); // Luis
System.out.println(nombres.tamanio()); // 2
```

Punto de control: la salida debe ser `Ana`, `Luis`, `2`.

```
C:\Estructura de Datos\LabColas_NoemiMataRuiz\src>javac colas\Main.java

C:\Estructura de Datos\LabColas_NoemiMataRuiz\src>java colas.Main
Entorno listo
Probar la cola
Sale: 10
Frente: 20
Tamaño: 3
¿Llena? true
Probar con genericos
Ana
Luis
2
```

Preguntas de análisis (Parte 2)

1. ¿Por qué es necesario `fin = null` cuando la cola queda vacía?

   Si no se hace, fin seguiría apuntando a un nodo que ya salió de la cola. La cola estaría vacía pero tendría un “último” elemento, lo cual no tiene sentido. Además, ese nodo no se podría borrar de la memoria.

2. ¿Qué ventaja tiene esta versión frente a la de arreglo? ¿Qué costo de memoria adicional implica?

   La ventaja es que no tiene límite: crece y se reduce según se necesite, sin llenarse nunca.
   El costo es que cada elemento se guarda en un nodo que además lleva una referencia al

siguiente, así que usa más memoria por elemento que un arreglo.

3. ¿Por qué usamos `T` en lugar de `int`?

Porque con T la misma clase sirve para cualquier tipo de dato: textos, números, objetos, etc. No hay que escribir una cola distinta para cada tipo, y Java revisa los tipos al compilar.

Parte 3. La interfaz `Queue` de Java (15 min)

Paso 3.1. Usar `Queue` con `LinkedList` y `ArrayDeque`

```
import java.util.ArrayDeque;
import java.util.LinkedList;
import java.util.Queue;

Queue<String> cola = new LinkedList<>();
// Prueba también: Queue<String> cola = new ArrayDeque<>();

cola.offer("Ana");
cola.offer("Luis");
cola.offer("María");

System.out.println(cola.peek()); // Ana (no la elimina)
System.out.println(cola.poll()); // Ana (la elimina)
System.out.println(cola.size()); // 2
System.out.println(cola); // [Luis, María]
```

Paso 3.2. Comparar métodos con excepción vs. valor especial

Con una cola **vacía**, ejecuta cada método y completa la tabla:

Método	¿Qué ocurrió? (excepción / null)
`remove()`	`remove()  -> excepción NoSuchElementException`
`poll()`	`poll()    -> null`
`element()`	`element() -> excepción NoSuchElementException`
`peek()`	`peek()    -> null`

Pista: los métodos `add`, `remove` y `element` lanzan excepción; `offer`, `poll` y `peek` devuelven un valor especial.

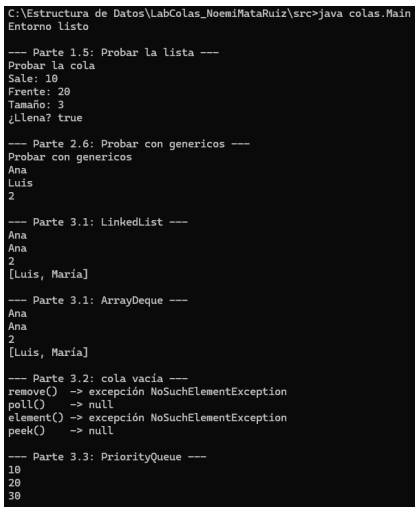
Paso 3.3. Cola de prioridad

```
Queue<Integer> pq = new java.util.PriorityQueue<>();
pq.offer(30);
pq.offer(10);
```

```
pq.offer(20);
System.out.println(pq.poll()); // ¿Qué valor sale primero?
System.out.println(pq.poll());
System.out.println(pq.poll());
```

Pregunta: ¿en qué se diferencia el orden de salida de una `PriorityQueue` respecto a una cola FIFO normal?

Salen 10, 20 y 30, aunque se metieron como 30, 10 y 20. En una cola FIFO normal los elementos salen en el orden en que llegaron. En una PriorityQueue siempre sale primero el de mayor prioridad, que por defecto es el número más pequeño.



Parte 4. Problema integrador: simulación de fila de banco (25 min)

Enunciado: simula una fila de banco con un solo cajero usando tu clase `ColaEnlazada<String>`. El programa debe mostrar este menú en un ciclo hasta que el usuario elija salir:

```
1. Llega un cliente
2. Atender al siguiente cliente
3. Ver quién sigue
4. Ver cantidad de clientes en espera
5. Salir
```

Paso 4.1. Estructura base

```
import java.util.Scanner;

public class Banco {
 public static void main(String[] args) {
 ColaEnlazada<String> fila = new ColaEnlazada<>();
 Scanner sc = new Scanner(System.in);
 int opcion;
```

```
 do {
 System.out.println("\n1. Llega un cliente");
 System.out.println("2. Atender al siguiente cliente");
System.out.println("3. Ver quién sigue");
 System.out.println("4. Ver cantidad en espera");
System.out.println("5. Salir");
 System.out.print("Opción: ");
 opcion = sc.nextInt();
 sc.nextLine();

 switch (opcion) {
 case 1:
 // TODO: pedir el nombre y encolarlo
                    break;
 case 2:
       // TODO: desencolar (validar que no esté vacía) break;
 case 3:
       // TODO: mostrar el frente (validar que no esté vacía) break;
 case 4:
 // TODO: mostrar el tamaño
 break;
 }
 } while (opcion != 5);
 }
}
```

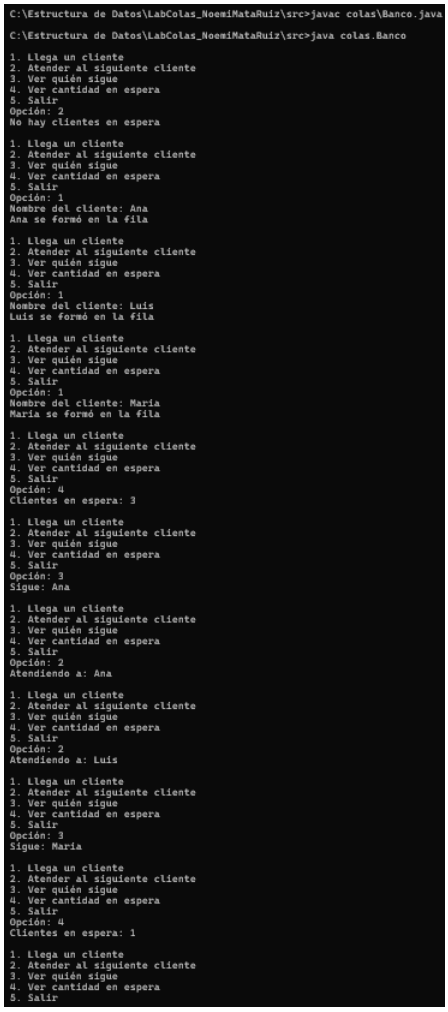
Paso 4.2. Completar los `TODO`

Implementa cada caso. **Importante:** antes de atender o consultar, verifica `estaVacia()` e imprime un mensaje amigable en lugar de dejar que el programa falle con una excepción.

Paso 4.3. Casos de prueba

Ejecuta esta secuencia y verifica el resultado esperado:

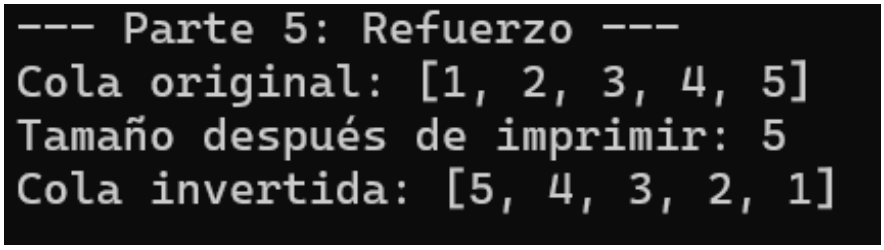
Acción	Resultado esperado
Opción 2 con fila vacía	Mensaje "No hay clientes en espera"
Llegan Ana, Luis y María	Tamaño = 3
Opción 3	Muestra "Ana"
Opción 2 (dos veces)	Atiende a Ana y luego a Luis
Opción 4	Tamaño = 1



Parte 5. Ejercicios de refuerzo (según el tiempo disponible)

Elige **al menos dos**:

1. **Nivel básico:** agrega a `ColaEnlazada` un método `imprimirCola()` que muestre todos los elementos del frente al final, sin destruir la cola.
2. **Nivel básico:** agrega un método `vaciar()` que deje la cola sin elementos.
3. Nivel intermedio: invierte una cola usando una pila (java.util.Stack o ArrayDeque como pila).
4. **Nivel intermedio:** implementa una cola usando **dos pilas**.
5. **Nivel avanzado:** usa una cola para recorrer un árbol binario por niveles (BFS).



6. Entregables

Sube al GitHub

- ☐ Código fuente: `ColaArreglo.java`, `ColaEnlazada.java`, `Banco.java` y `Main.java`.
- ☐ Código de los ejercicios de refuerzo elegidos.
- ☐ Un documento breve (máx. 1 página) con:
    - o Las respuestas a las preguntas de análisis.
    - o La tabla completada del Paso 3.2.
    - o Capturas de pantalla de las pruebas ejecutadas.

7. Rúbrica de evaluación

Criterio	Puntos
`ColaArreglo` funciona correctamente (incluido el giro circular)	20
`ColaEnlazada` genérica funciona correctamente	25
Uso correcto de `Queue` y tabla del Paso 3.2 completa	15
Simulación del banco con validaciones	20
Ejercicios de refuerzo	10
Respuestas de análisis, orden y claridad del código	10
Total	**100**

8. Errores comunes a evitar

- Olvidar `% capacidad` al avanzar los índices en la cola circular.
- No actualizar `fin = null` al vaciar la cola enlazada.
- No validar `estaVacia()` antes de `desencolar` o `frente`.
- Confundir `peek()` (consulta) con `poll()` (elimina).
- Olvidar `sc.nextLine()` después de `nextInt()` al leer texto con `Scanner`.

9. Cierre y reflexión

Responde brevemente:

1. ¿En qué situaciones elegirías una cola con arreglo y en cuáles una enlazada?

   Usaría la cola con arreglo cuando sé cuántos elementos habrá como máximo, porque gasta menos memoria. Usaría la enlazada cuando no sé cuántos elementos van a llegar, porque crece sin límite.

2. Menciona dos ejemplos de la vida real o de sistemas informáticos donde se usen colas.

   La cola de impresión, donde los documentos se imprimen en el orden en que se mandaron, y la fila de clientes en un banco o en un call center.

3. ¿Qué fue lo más difícil de la práctica y cómo lo resolviste?

   Lo más difícil fue entender cómo da la vuelta el arreglo circular con el operador %, y también un error que me salió por pegar código fuera del método main. Lo resolví siguiendo los valores de frente y fin en papel, y revisando dónde abría y cerraba cada llave